

AI Usage Note – KB Gap Finder

What AI Helped With

1. Topic Clustering Strategy

AI helped identify the best approach for grouping similar support tickets into meaningful clusters. Various clustering techniques were evaluated, and a lightweight solution suitable for a hackathon environment was selected.

2. Knowledge Gap Detection Design

AI suggested comparing recurring ticket clusters against existing Knowledge Base articles using semantic similarity techniques. This enabled the system to automatically identify topics that lack documentation.

3. Embedding and Similarity Search

AI recommended using text embeddings and cosine similarity to measure how closely ticket clusters match existing KB articles. This improved the accuracy of gap detection beyond simple keyword matching.

4. KB Article Generation Prompt Design

AI assisted in creating prompts that generate structured Knowledge Base articles containing:

- Title
- Problem Statement
- Possible Causes
- Resolution Steps
- FAQs
- Additional References

5. Agent Workflow Design

AI helped design the end-to-end agent workflow:

1. Read support tickets
2. Generate embeddings
3. Cluster similar issues
4. Compare with KB repository
5. Detect documentation gaps
6. Generate draft KB articles
7. Present recommendations

6. Streamlit Dashboard Development

AI generated code suggestions for:

- CSV upload functionality
- Interactive dashboards
- Cluster visualization
- Gap analysis reporting
- KB article preview screens

7. Database Design

AI helped create the database schema for storing:

- Support Tickets
- Clusters
- Existing KB Articles
- Gap Analysis Results
- Generated KB Drafts

What AI Got Wrong

Issue	What Happened	Fix
Over-Clustering	AI initially created too many small clusters for similar issues	Adjusted clustering parameters and similarity thresholds
Duplicate KB Recommendations	Similar ticket groups generated multiple recommendations for the same topic	Added cluster deduplication logic
Generic KB Articles	Initial drafts lacked technical details	Improved prompts with ticket examples and context
Incorrect Similarity Thresholds	Some valid KB articles were incorrectly marked as gaps	Tuned cosine similarity threshold through testing
Large Ticket Processing	AI-generated implementation was slow on larger datasets	Optimized preprocessing and caching

Best Prompts Used

1. Clustering Prompt

```
Analyze the support ticket dataset and group semantically similar issues into meaningful clusters. Provide cluster names and common themes.
```

2. Gap Detection Prompt

```
Compare recurring support ticket clusters with existing Knowledge Base articles and identify clusters that do not have adequate documentation.
```

3. KB Article Generation Prompt

Generate a professional Knowledge Base article using the provided ticket cluster. Include title, problem statement, causes, resolution steps, FAQs, and references.

4. Agent Workflow Prompt

Design an AI agent workflow that identifies recurring support issues, checks KB coverage, and recommends new KB articles using free and open-source tools.

5. Streamlit Development Prompt

Build a Streamlit dashboard that uploads support tickets, performs clustering, detects KB gaps, and displays generated Knowledge Base article recommendations.

AI Tools Used

- ChatGPT (Architecture Design, Documentation, Prompt Engineering)
- Open AI Codex (Code Completion and Refactoring)
- Ollama (Local LLM for KB Article Generation)
- Open-Source Python Libraries (Scikit-Learn, Pandas, Streamlit)